

SBNTS / AmberWear — Security Hardening

This document describes the security issues that were found in the store and how they were fixed. The app is a single static page (`SBNTS.html`) backed by **Supabase** (Auth + Postgres with Row Level Security). There is no custom application server, so the *server side* is Supabase: its Auth service, the RLS policies, and the SQL functions/triggers in `files/schema.sql`.

Fixing this properly required changes in **both** places:

1. `SBNTS.html` — the front-end.
 2. `files/schema.sql` — the database rules (**you must re-run this in the Supabase SQL Editor** for the server-side fixes to take effect).
 3. A few **Supabase Dashboard** toggles (listed at the bottom).
-

1. Locally stored session tokens / identity ✓ fixed

Before: the whole user object — *including* `role` — was written to `localStorage` (`aw_user`) and read back on load. Anyone could open the console and run `localStorage.setItem('aw_user', '{"role": "admin"}')` to unlock the admin UI, and identity survived independently of any real session.

After:

- `user` now lives **only in memory** and is rebuilt exclusively from the verified Supabase session via `refreshUser()`.

- `save()` no longer persists identity or role — only cart/product UI state.
- Any legacy `aw_user` / `aw_role` values are purged from `localStorage` on load.
- The real session token is still managed by the Supabase SDK (it uses an `sb-*` storage key with refresh-token rotation) — that is the SDK's designed, scoped mechanism, and it is *not* something the app hand-rolls.

2. Admin checks were done on the client ✓ fixed

Before: `openAdminModal()` and the Admin nav button trusted the cached `user.role`. Product create/update/delete relied on the UI to hide the panel.

After:

- A new **server-side** RPC `public.is_admin()` (SECURITY DEFINER) is the single source of truth. `openAdminModal()` calls `fetchIsAdmin()` and refuses to open otherwise.
- Product writes are still ultimately enforced by **RLS** on the `products` table (Admins can insert/update/delete now use `public.is_admin()`), so even a scripted request with the anon key cannot modify inventory without a real admin session. The button visibility is now purely cosmetic.

3. Privilege escalation via RLS (server side) ✓ fixed

Before: the `profiles` UPDATE policy allowed a user to update *any* column of their own row — including `role` — so a customer could self-promote to `admin` with a single anon-key request. This was the most severe issue.

After:

- The UPDATE policy is scoped with both `using` and `with check`.
- A `before update` trigger `prevent_role_change()` blocks any change to `role` unless the caller is already an admin. The service role / SQL editor (no `auth.uid()`) is still allowed, which is how you promote the first admin.
- New signups always get `role='customer'` (enforced in `handle_new_user()`), and the client no longer inserts profile rows with a chosen role.

4. Admin OTP (second factor) ✓ added

Admins now pass a **two-step** sign-in:

1. Email + password (verified by Supabase Auth).
2. A **6-digit one-time code emailed** to the admin (`signInWithOtp` → `verifyOtp`). The admin UI is not granted until the OTP is verified *and* `is_admin()` is re-confirmed server-side. The `onAuthStateChange` handler deliberately withholds the UI while an admin is mid-OTP.

This gives a materially more secure admin login than a password alone, using only Supabase's built-in email OTP (no extra service required).

Optional upgrade: Supabase also supports **TOTP authenticator-app MFA** (`supabase.auth.mfa.*`). If you later want app-based codes instead of email, enable MFA in the dashboard and swap the OTP step for an MFA challenge.

5. Rate limiting for login & password resets ✓ added

A database-backed limiter that **cannot be bypassed** by clearing the browser store or scripting the anon key:

- Table `public.auth_rate_limits` + `check_and_record_attempt()` / `clear_rate_limit()` (both SECURITY DEFINER; table has no direct access).
- Enforced budgets:
 - **Login:** 5 attempts / 15 min per email.
 - **Password reset:** 3 emails / hour per email.
 - **Register:** 5 / hour per email. **OTP send/verify:** 5–6 / 15 min.
- Successful login clears the counter. This complements Supabase Auth's own built-in rate limits (see dashboard settings below).

6. Password-strength checks ✓ added

- `register()` requires **10+ chars with upper- & lower-case, a number and a symbol**, plus a confirm-password match, before calling `signUp`.
- A live strength meter (`updatePwMeter`) gives real-time feedback.
- Set a matching **minimum password policy in the Supabase dashboard** so the rule is also enforced server-side (client checks are UX, not a security boundary).

7. Removed the insecure "offline mode" ✓ fixed

Before: `hasSupabaseConfig` compared each config value *to itself*, so it was always `false`; `supabaseClient` became `null` and `doLogin()` logged **anyone in with no password check** ("Signed in locally").

After: the config check validates a real Supabase URL + anon key, and the passwordless fallback is gone. If the backend isn't configured, auth is simply unavailable rather than silently insecure.

8. Other touch-ups

- `esc()` applied to the username shown in the header (defence-in-depth XSS).
 - Login errors are generic ("Incorrect email or password"), and password reset shows a neutral message, to avoid leaking which emails are registered.
-

What YOU must do in Supabase

1. **Run the updated `files/schema.sql`** in the SQL Editor (it is safe to re-run — it drops/recreates policies, functions and triggers).
2. **Authentication → Providers → Email:** ensure **Email OTP** is enabled (needed for the admin second factor) and keep "Confirm email" on.
3. **Authentication → Policies / Password:** set a **minimum password length of at least 10** and require character classes if available, to mirror the client rule.
4. **Authentication → Rate Limits:** keep/lower Supabase's built-in limits for sign-in, OTP and password-reset emails (defence in depth on top of the `auth_rate_limits` table).
5. **Promote your admin** (service-role context bypasses the role-change guard):

```
update public.profiles set role = 'admin'
where id = (select id from auth.users where email = '');
```

6. **Rotate the anon key** shown in the file if the project was ever public. The anon key is safe to expose *only because* RLS is correct — which it now is.